

# COMP3320/6464 Week 2, Lecture 2

Rhys Hawkins

Semester 2, 2024

# Yesterday

- Floating Point
- Approximation and Rounding Error
- Catastrophic Cancellation

# Today

- Simple Optimisation Guidelines

Next week

- Cache
- Pipeline and Super Scalar Architecture
- Loop analysis and Unrolling

# General rules for good HPC scientific code

- Write modular code
  - No more than one page per function module
  - Makes it easier to
    - understand code
    - profile code
    - test/verify code
  - Intensive Code that is called together should be in the same module [C/C++]

# Optimisation Guidance

*Premature optimisation is the root of all evil - Donald Knuth*

- Above all: working code first

Optimisation: Iterate

- Benchmark/Profile your code (for both performance and correctness)
- Make a single improvement/optimisation
- Has this improved performance? Is it still correct?

Why?

- Combinations of different optimisations can affect each other
- Different CPUs/compilers operate differently
- Optimisations may introduce subtle bugs

# Avoid Dynamic Memory Allocation

- Dynamic memory allocation is an expensive operation
- Especially for many different sized allocations
- Prefer pre-allocated buffers that have enough for maximum size

```
float compute_nterm_sum(int N,
                        float x,
                        float *C)
{
    int i;
    float *terms = malloc(sizeof(float) * N);
    for (i = 0; i < N; i++) {
        terms[i] = x * C[i];
        s = s + terms[i];
    }
    free(terms);
    return s;
}

...

for (int i = 0; i < 1000000; i++) {
    s = compute_nterm_sum(random() % 10 + 1, x, C);
}
```

# Avoid Dynamic Memory Allocation

- Dynamic memory allocation is an expensive operation
- Especially for many different sized allocations
- Prefer pre-allocated buffers that have enough for maximum size

```
float compute_nterm_sum(int N,
                        float x,
                        float *C)
{
    int i;
    float terms[10];
    assert(N <= 10);

    for (i = 0; i < N; i++) {
        terms[i] = x * C[i];
        s = s + terms[i];
    }

    return s;
}

...

for (int i = 0; i < 1000000; i++) {
    s = compute_nterm_sum(random() % 10 + 1, x, C);
}
```

# Do less work!

```
int i;  
int flag = 0;  
for (i = 0; i < N; i++) {  
    if (complex_function(A[i]) < THRESHOLD) {  
        flag = 1;  
    }  
}
```

- Only care if one result is below threshold so we can exit early



# Do less work!

```
int i;  
int flag = 0;  
for (i = 0; i < N; i++) {  
    if (complex_function(A[i]) < THRESHOLD) {  
        flag = 1;  
        break;  
    }  
}
```

- Terminate early if possible
- Caution: this applies where `complex_function` is expensive relative to the size of the loop

# Avoid expensive operations: refactorise

```
int i;  
float s = 1.0;  
  
for (i = 0; i < N; i++) {  
    s = s * exp(A[i]);  
}  
  
return s;
```

- Many operations (sin, cos, exp, log) can be expensive

# Avoid expensive operations: refactorise

```
int i;  
float s = 0.0;  
  
for (i = 0; i < N; i++) {  
    s = s + A[i];  
}  
  
return exp(s);
```

- Many operations (sin, cos, exp, log) can be expensive
- Using  $\exp A \times \exp B = \exp(A + B)$
- Sometimes factorisation of the problem can eliminate or reduce calls to special functions
- Caution: floating point accuracy means that some refactors may produce different results (test)

# Avoid expensive operations: a common one

```
int i;  
float s = 0.0;  
  
for (i = 0; i < N; i++) {  
    s = s + pow(A[i], 2);  
}  
  
return sqrt(s);
```

- The pow function is generally slow

# Avoid expensive operations: a common one

```
int i;  
float s = 0.0;  
  
for (i = 0; i < N; i++) {  
    s = s + A[i]*A[i];  
}  
  
return sqrt(s);
```

- The pow function is generally slow
- Multiplication is only 3-4 cycles and can be vectorised
- Even larger integer powers are generally better implemented by multiplication rather than calls to pow.

# Avoid expensive operations: lookup tables

```
int i;  
int T = 10;  
for (i = 0; i < N; i++) {  
    A[i] = sin(2.0 * M_PI * (float)i / (float)T);  
}
```

- Sometimes we only need to compute a few discrete values of special functions

# Avoid expensive operations: lookup tables

```
int i;
static const int T = 10;
float lookup[T];

for (int i = 0; i < T; i++) {
    lookup[i] = sin(2.0 * M_PI * (float)i / (float)T);
}

for (i = 0; i < N; i++) {
    A[i] = lookup[i % T];
}
```

- Sometimes we only need to compute a few discrete values of special functions
- It is sometimes quicker to use a lookup table
- Caution: large lookup tables start to cost more than the function itself due to memory/cache (always time your code)

# Avoid expensive operations: division

```
void f1(int n,  
        double *a,  
        double *b,  
        double *c,  
        double *d,  
        double *y,  
        double *z)  
{  
    int i;  
    for (i = 0; i < n; i++) {  
        y[i] = a[i] / b[i];  
        z[i] = c[i] / d[i];  
    }  
}
```

- On many CPUs + and \* are cheaper than divide



# Avoid expensive operations: division

```
void f1(int n,  
        double *a,  
        double *b,  
        double *c,  
        double *d,  
        double *y,  
        double *z)  
{  
    int i;  
    double tmp;  
    for (i = 0; i < n; i++) {  
        tmp = 1.0/(b[i] * d[i]);  
        y[i] = a[i] * d[i] * tmp;  
        z[i] = c[i] * b[i] * tmp;  
    }  
}
```

- On many CPUs + and \* are cheaper than divide
- This unintuitive optimisation can sometimes improve performance
- More recent CPUs have division closer to multiplication in terms of cycles

# Shrink the working set (Size in Memory)

```
void f1(int n, double *a)
{
    int i, j;
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            a[i*n + j] = i + j;
        }
    }
}
```

- Sometimes we don't need double precision

# Shrink the working set (Size in Memory)

```
void f1(int n, float *a)
{
    int i, j;
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            a[i*n + j] = i + j;
        }
    }
}
```

- Sometimes we don't need double precision
- By changing to float, we can sometimes double performance
- Caution: accuracy
- Caution: mixing floats, doubles, integers

# Elimination of Common Subexpressions

```
for (i = 0; i < N; i++) {  
    A[i] = A[i] + s + r*sin(x);  
}
```

- Move constants outside of loops

# Elimination of Common Subexpressions

```
float tmp = s + r*sin(x);  
for (i = 0; i < N; i++) {  
    A[i] = A[i] + tmp;  
}
```

- Move constants outside of loops
- Often performed by compiler optimisations

# Reduce/Eliminate branches in loops

```
for (j = 0; j < N; j++) {  
    for (i = 0; i < N; i++) {  
        if (i > j) {  
            sign = 1.0;  
        } else if (j > i) {  
            sign = -1.0;  
        } else {  
            sign = 0.0;  
        }  
  
        C[j] = C[j] + sign * A[i][j] * B[i];  
    }  
}
```

- `if` statements in loops can cause performance issues, particularly if an outcome of an `if` is random and 50:50

# Reduce/Eliminate branches in loops

```
for (j = 0; j < N; j++) {  
    for (i = j + 1; i < N; i++) {  
        C[j] = C[j] + A[i][j] * B[i];  
    }  
}  
for (j = 0; j < N; j++) {  
    for (i = 0; i < j; i++) {  
        C[j] = C[j] - A[i][j] * B[i];  
    }  
}
```

- `if` statements in loops can cause performance issues, particularly if an outcome of an `if` is random and 50:50
- Branch (mis)-prediction can cause wasted operations
- Removing `if`'s from loops can improve performance
- Caution: dynamic termination conditions in loops can also cause performance issues

# Becareful of hidden conversions

C

```
float f(float x) {  
    return 3*x;  
}
```

Assembly (approx. with -O3)

```
mulss xmm0, 3.0f
```



# Becareful of hidden conversions

C

```
float f(float x) {  
    return 3.1*x;  
}
```

Assembly (approx. with -O3)

```
cvtss2sd    xmm0, xmm0  
mulsd       xmm0, 3.0d  
cvtisd2ss   xmm0, xmm0
```

# Becareful of hidden conversions

C

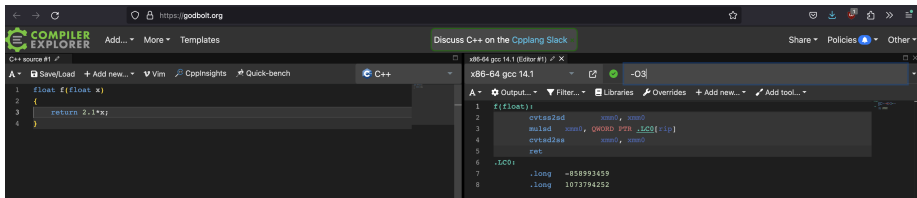
```
float f(float x) {  
    return 3.1f*x;  
}
```

Assembly (approx. with -O3)

```
mulss    xmm0, 3.0f
```

- In C, floating point values default to double precision.
- Also note this applies to the use of math functions such as `sin` and `sinf`
- Mixing integers and floating point requires care as well but is generally ok, e.g. `time 2` is often optimized as an addition operation.

# Godbolt for quickly checking code snippets for issues



<https://godbolt.org>

- My preferred way to see quickly what code will be generated
- Other options include
  - Compiler flags to output assembly: `gcc -S -masm=intel f.c`
  - Using objdump after compilation: `objdump -D -M intel f.o`

# Loop unrolling

```
for (i = 0; i < N; i++) {  
    A[i] = B[i] * C[i] + D[i];  
}
```

- For some loops with floating point operations, we can unroll the loop for marginal performance improvement

# Loop unrolling

```
for (i = 0; i < N; i += 2) {  
    A[i] = B[i] * C[i] + D[i];  
    A[i + 1] = B[i + 1] * C[i + 1] + D[i + 1];  
}
```

- For some loops with floating point operations, we can unroll the loop for marginal performance improvement
- In many cases, the effect is marginal
- Caution: Sometimes it can be detrimental (increases code size)
- In lectures starting next week, we will look more indepth at loop performance improvements.

# Summary

- Several simple ways that may help improve performance
- There is no one guaranteed way to improve performance
  - Different CPUs behave differently
  - Different compilers may perform other optimisations that make things better/worse
  - One or more optimisations may interfere with each other
  - Understand various compiler optimisation flags and some of the things they do for you
- Important: Measure the impact of your changes
  - Has it improved performance?
  - Does the code still produce the correct result?